极客时间

成本控制实战

工程思想：Token 经济学

第 2 周 · 第 8 节

讲师：黄佳

极客时间黄佳老师出品

极客时间

极客时间

灵魂拷问

用 AI 做个人知识库，一个月至少要花多少钱？

答案：可以控制在 ￥5–10 /月

极客时间

本周工程思想

Token 经济学
每个 token 都有成本，
成本从第一天就塑造架构

不是“先做完再优化”，而是“成本意识融入设计”

Token 经济学三个核心概念

极客时间

概念	含义	例子
Token Debt （Token 债务）	因架构设计不当而产生的 持续性 token 浪费	每次调用都发送完整上下文， 不做裁剪——债务随调用量 线性增长
Token Leverage （Token 杠杆）	用少量 token 撬动大量 高质量输出的能力	精心设计的 system prompt 用 200 tokens 指令 省掉 2000 tokens 的纠错
Token Refactoring Dividend （Token 重构红利）	重构 prompt/架构后 获得的长期 token 节省	将万能 prompt 拆成 3 个专用 prompt， 总 token 消耗降 60%

Token 基础知识

- **什么是 Token?** — 大模型处理文本的最小单位，不是字也不是词
-
- **中文：** 1 个汉字 ≈ 1.5–2 tokens（取决于模型的分词器）
- 「人工智能」4 个字 ≈ 6–8 tokens
- **英文：** 1 个单词 ≈ 1–1.5 tokens
- “artificial intelligence” 2 词 ≈ 3–4 tokens

- **输入 Token vs 输出 Token：**
- 输入 Token — 你发给模型的内容（system prompt + 用户消息）
- 输出 Token — 模型生成的回复内容
- 关键：输出 token 通常比输入 token 贵 2–4 倍！
-
- **为什么要了解 Token?**
- 因为 API 按 token 计费，不按次数计费
- 同样调一次 API，100 tokens 和 10000 tokens 差 100 倍

极客时间黄佳老师出品

国产模型价格矩阵（2026 年 3 月）

极客时间

模型	输入价格	输出价格	定位	适用场景
GLM-4-Flash	免费	免费	入门级	简单分类/格式化
DeepSeek-V3	¥1/M tokens	¥2/M tokens	性价比之王	日常分析/总结
Qwen-Plus	¥4/M tokens	¥12/M tokens	中高档	复杂推理/长文
Kimi (Moonshot)	¥12/M tokens	¥12/M tokens	长上下文	超长文档处理
DeepSeek-R1	¥4/M tokens	¥16/M tokens	推理增强	数学/逻辑推理
GPT-4o（参考）	¥45/M tokens	¥135/M tokens	国际顶级	（成本对比参考）

成本速算公式

- **基础公式：**
- 单次成本 = (输入 tokens + 输出 tokens) × 单价

- **实际案例 — 用 DeepSeek-V3 分析一篇技术文章：**
- 输入：system prompt 500 tokens + 文章内容 2000 tokens = 2500 tokens
- 输出：摘要 + 亮点 + 评分 ≈ 500 tokens
- 成本 = 2500 × ¥0.000001 + 500 × ¥0.000002 = ¥0.0035
-

- **一句话记忆法：**
- 用 DeepSeek 做一次完整分析 ≈ ¥0.003–0.005
- 约等于三分钱的千分之一
- 换句话说：分析 1000 篇文章才花 3–5 块钱

- **成本直觉：** 单次调用极便宜 → 关键是控制调用次数和 token 量

极客时间黄佳老师出品

三档模型路由策略 — 把钱花在刀刃上

极客时间

第 1 档 · 零成本	第 2 档 · 低成本	第 3 档 · 按需使用
搜索 / 采集	分析 / 总结	决策 / 审核
纯 API 调用 GitHub API HN API RSS Feed	**便宜模型** DeepSeek-V3 GLM-4-Flash	**高档模型** Qwen-Plus DeepSeek-R1
¥0/次	¥0.003/次	¥0.02-0.05/次

极客时间

Pipeline 成本逐步分析

极客时间

步骤	操作	模型	Token 消耗	单次成本
Step 1	GitHub API 采集	无（纯 API）	0	¥0
Step 2	HN API 采集	无（纯 API）	0	¥0
Step 3	内容分析 + 摘要	DeepSeek-V3	~3000	≈ ¥0.005
Step 4	趋势总结 + 评分	DeepSeek-V3	~2000	≈ ¥0.003
合计	一次完整流水线	—	~5000	≈ ¥0.008

月度成本估算

- **日常运行（CI/CD 自动触发）：**
- 每天 1 次完整流水线：¥0.008 × 30 天 = ¥0.24/月

- **手动调试和测试：**
- 开发阶段反复测试 Skill 效果：约 ¥2–3/月
- 偶尔手动触发额外分析：约 ¥1–2/月
- **综合估算：**
- 保守估计 ¥3–5/月 | 宽松估计 ¥5–10/月
- **对比参考：**
- ChatGPT Plus 订阅 ≈ ¥140/月
- Claude Pro 订阅 ≈ ¥140/月
- 我们的知识库系统 ≈ ¥5–10/月（功能更定制，成本低 15–30 倍）

极客时间黄佳老师出品

model_client.py — 成本意识设计

极客时间

project/ai-knowledge-base/model_client.py | python

```python
"""模型客户端 — 支持多模型切换 + 成本感知"""
import os
from openai import OpenAI  # 国产模型普遍兼容 OpenAI 接口

# 模型配置表 — 成本从低到高
MODEL_CONFIGS = {
    "free":     {"model": "glm-4-flash",
                "base_url":
"https://open.bigmodel.cn/api/paas/v4/",
                "price_in": 0, "price_out": 0},
    "cheap":    {"model": "deepseek-chat",
                "base_url": "https://api.deepseek.com",
                "price_in": 1, "price_out": 2},  # ¥/M tokens
    "mid":      {"model": "qwen-plus",
                "base_url":
"https://dashscope.aliyuncs.com/compatible-mode/v1",
                "price_in": 4, "price_out": 12},
    "premium":  {"model": "deepseek-reasoner",
                "base_url": "https://api.deepseek.com",
                "price_in": 4, "price_out": 16},
}
```

project/ai-knowledge-base/model_client.py | python

```python
def call_model(prompt, tier="cheap", max_tokens=1024,
temp=0.3):
    """调用模型 — tier 参数决定用哪档模型"""
    cfg = MODEL_CONFIGS[tier]
    client = OpenAI(base_url=cfg['base_url'],
                    api_key=os.getenv(f'{tier.upper()}_API_KEY'))
    resp = client.chat.completions.create(
        model=cfg['model'],
        messages=[{'role': 'user', 'content': prompt}],
        max_tokens=max_tokens, temperature=temp)
    return resp
```

代码展示 — Token 消耗统计

极客时间

project/ai-knowledge-base/model_client.py | python

```python
# 在 model_client.py 中加入 token 统计

class CostTracker:
    """追踪每次调用的 token 消耗和成本"""
    def __init__(self):
        self.total_input_tokens = 0
        self.total_output_tokens = 0
        self.total_cost = 0.0  # 单位：元
        self.calls = []        # 调用记录

    def record(self, usage, tier):
        """从 API 响应的 usage 字段提取 token 数据"""
        cfg = MODEL_CONFIGS[tier]
        in_tokens = usage.prompt_tokens
        out_tokens = usage.completion_tokens
        cost = (in_tokens * cfg['price_in']
                + out_tokens * cfg['price_out']) / 1_000_000
```

project/ai-knowledge-base/model_client.py | python

```python
        self.total_input_tokens += in_tokens
        self.total_output_tokens += out_tokens
        self.total_cost += cost
        self.calls.append({
            'tier': tier, 'in': in_tokens,
            'out': out_tokens, 'cost': f'¥{cost:.6f}'})

    def summary(self):
        print(f"总调用 {len(self.calls)} 次")
        print(f"输入 {self.total_input_tokens} tokens")
        print(f"输出 {self.total_output_tokens} tokens")
        print(f"总成本 ¥{self.total_cost:.4f}")
```

代码展示 — 成本计算器

极客时间

project/ai-knowledge-base/cost_calculator.py | python

```python
# cost_calculator.py — 不同场景的成本对比计算

scenarios = {
    "每日自动采集+分析": {
        "calls_per_day": 2,          # 2 次模型调用
        "avg_tokens_per_call": 2500, # 平均 token
        "tier": "cheap",             # DeepSeek-V3
    },
    "手动深度分析": {
        "calls_per_day": 5,
        "avg_tokens_per_call": 4000,
        "tier": "cheap",
    },
    "全部用高档模型": {
        "calls_per_day": 7,
        "avg_tokens_per_call": 3000,
        "tier": "mid",               # Qwen-Plus
    },
}
```

project/ai-knowledge-base/cost_calculator.py | python

```python
for name, s in scenarios.items():
    cfg = MODEL_CONFIGS[s['tier']]
    daily = (s['calls_per_day'] * s['avg_tokens_per_call']
             * (cfg['price_in'] + cfg['price_out'])
             / 2_000_000)  # 粗算：输入输出各半
    monthly = daily * 30
    print(f"{name}: ¥{daily:.4f}/天 ≈ ¥{monthly:.2f}/月")

# 输出示例：
# 每日自动采集+分析: ¥0.0075/天 ≈ ¥0.23/月
# 手动深度分析:     ¥0.0300/天 ≈ ¥0.90/月
# 全部用高档模型:    ¥0.1680/天 ≈ ¥5.04/月
```

极客时间黄佳老师出品

缓存策略 — 减少重复消耗

- **原则：** 相同的输入不要重复调用模型

- **AGENTS.md 缓存优势：**
- AGENTS.md 内容稳定 → system prompt 不变 → 模型可复用 KV Cache
- DeepSeek 等模型支持 prompt caching，重复的前缀部分享受折扣
- **数据采集层面：**
- 增量采集 vs 全量采集 — 只处理新数据
- 文件指纹（hash）判断内容是否变化 — 没变化就跳过
- 上次采集的 ID 列表 → 这次只分析新增的
- **结果缓存：**
- 分析结果写入文件 → 下次先检查缓存文件是否存在
- 缓存有效期：24 小时（每日采集场景）
- **经验数据：** 缓存策略可减少 40–70% 的 token 消耗

极客时间黄佳老师出品

Prompt 优化 — 减少 Token 消耗的 4 个技巧

极客时间

- **技巧 1：精简 system prompt**
- 删除冗余描述 → 2000 tokens 压缩到 500 tokens
- 用结构化指令替代长段落 → 每次调用省 1500 tokens

- **技巧 2：约束输出格式**
- 要求输出 JSON 而非长文 → 输出 token 减少 60–80%
- “只输出 JSON，不要解释” → 一句话省几百 tokens

- **技巧 3：批量处理 vs 逐条处理**
- 10 条数据一次性发送 → 省 9 次 system prompt 开销
- 权衡：batch 太大可能降低质量 → 建议 5–10 条/批

- **技巧 4：分层提问**
- 第一轮：用免费模型做粗筛（符合/不符合）
- 第二轮：只对通过粗筛的数据用付费模型做深度分析

极客时间

对比实验：全高档 vs 混合路由

极客时间

方案 A：全部用 Qwen-Plus

Step 1–2 采集：Qwen-Plus 做数据清洗
Step 3 分析：Qwen-Plus 深度分析
Step 4 总结：Qwen-Plus 趋势总结

每日成本：≈ ¥0.24
月成本：≈ ¥7.20

优点：质量统一
缺点：采集步骤浪费模型能力
成本是方案 B 的 10 倍

方案 B：三档混合路由

Step 1–2 采集：纯 API（零成本）
Step 3 分析：DeepSeek-V3（便宜）
Step 4 总结：DeepSeek-V3（便宜）

每日成本：≈ ¥0.008
月成本：≈ ¥0.24

优点：成本极低
优点：每步选最合适的工具
实际质量差异：<5%

预算控制 Hook — 超限自动告警/停止

极客时间

project/ai-knowledge-base/hooks/budget_guard.py | python

```python
# hooks/budget_guard.py — 预算守卫 Hook

import json
from pathlib import Path

BUDGET_FILE = Path('data/budget.json')
DAILY_LIMIT = 0.50     # 每日上限 ¥0.50
SINGLE_LIMIT = 0.10    # 单次上限 ¥0.10

def check_budget(cost: float) -> bool:
    """调用前检查预算"""
    budget = json.loads(BUDGET_FILE.read_text())
    today = budget.get('today_cost', 0)

    # 单次超限 → 告警但继续
    if cost > SINGLE_LIMIT:
        print(f"[WARN] 单次成本 ¥{cost:.4f} 超过上限
¥{SINGLE_LIMIT}")
```

project/ai-knowledge-base/hooks/budget_guard.py | python

```python
    # 日累计超限 → 停止执行
    if today + cost > DAILY_LIMIT:
        print(f"[STOP] 日累计 ¥{today + cost:.4f} 将超过
¥{DAILY_LIMIT}")
        return False  # 拒绝执行

    # 更新预算记录
    budget['today_cost'] = today + cost
    BUDGET_FILE.write_text(json.dumps(budget, indent=2))
    return True  # 允许执行

# 在 pipeline.py 中使用：
# if not check_budget(estimated_cost):
#     print('预算超限，跳过本次调用')
#     sys.exit(1)
```

V2 完整架构图 — Pipeline + Hooks + CI/CD + 成本控制

GitHub Actions (CI/CD)
每日定时触发 cron: 0 8 * * *

Step 1–2
采集（API）
¥0

Step 3
分析
¥0.005

Step 4
总结
¥0.003

Hooks: budget_guard +quality_check + log

CostTracker：token 统计 + 成本报表

第 2 周总结：4 节课回顾

极客时间

- **第 5 节 · Hooks（反馈循环）**
- 事件驱动自动化 → 代码一变就自动响应
- 工程思想：反馈驱动循环

- **第 6 节 · MCP（数据连接）**
- 外部工具协议 → Agent 接入 API / 数据库 / 第三方服务
- 关键：标准化接口，即插即用

- **第 7 节 · CI/CD（自动触发）**
- GitHub Actions → pipeline 每天自动跑，零人工干预
- 关键：cron 定时 + secrets 管理 + artifact 存储

- **第 8 节 · 成本控制（Token 经济学）**
- 三档模型路由 + 缓存策略 + 预算 Hook
- 工程思想：成本从第一天就塑造架构

极客时间

本节作业

极客时间

任务 1：加入 Token 消耗统计

在 model_client.py 中集成 CostTracker 类。每次调用模型后自动记录 token 消耗和成本。pipeline 跑完后打印成本汇总报告。验证：跑一次完整 pipeline，检查报告数据是否合理。

任务 2：实现模型路由策略

修改 pipeline.py，不同步骤使用不同 tier 的模型。采集步骤不调用模型（纯 API）；分析步骤用 cheap 档（DeepSeek-V3）；如有审核步骤用 mid 档。验证：检查不同步骤是否调用了正确的模型。

任务 3：全高档 vs 混合路由对比实验

用相同的数据分别跑两套方案：A 全用 Qwen-Plus，B 混合路由。对比两个维度：(1) 成本差异 (2) 输出质量差异。写一份简短对比报告——这就是你选择路由策略的数据依据。

任务 4：提交 V2 完整项目

整合本周所有成果：pipeline.py + hooks/ + .github/workflows/ + model_client.py（含 CostTracker）+ budget_guard.py。确保 CI/CD 能自动触发、成本有追踪、预算有保护。这就是 V2 的完整交付物。

极客时间黄佳老师出品

工程思想总结卡 — Week 2 双卡

极客时间

卡 1：反馈驱动循环

来自：第 5 节 Hooks

核心理念：
系统行为应该由事件驱动，
而不是由人工干预驱动

三要素：
- 触发条件（事件）
- 执行动作（Hook）
- 反馈修正（循环）

应用：质量检查 → 格式化 → 日志

卡 2：Token 经济学

来自：第 8 节 成本控制

核心理念：
每个 token 都有成本，
成本从第一天就塑造架构

三要素：
- 模型分档（三档路由）
- 消耗测量（CostTracker）
- 预算保护（budget Hook）

应用：¥5–10/月 运行知识库系统

本章小结

极客时间

成本控制：token 用量 × 单价 = 实际开销

三道防线：预算上限 / 模型降级 / 熔断机制

V2 完整交付：Pipeline + Hooks + CI/CD + 成本控制

V1→V2 跨越：从编码自动化到采集自动化

下一节：第 9 节 · 多 Agent 设计模式

极客时间黄佳老师出品